pois, voltamos à merda da projecção

Aula 08

Today:

- We're back to circuits and formulas
- We will see the relationship between the class of functions computable by polynomial-size circuits and P
- We will prove lower-bounds for formulas

P/poly

Let me define the following class:

Definition. Let $POLYSIZE$ denote the class of functions $F : \{0,1\}^* \to \{0,1\}$, such that there exists a $c \in \mathbb{N}$ for which

$$\forall n \qquad SIZE(F_n) = O(n^c).$$

Where $F_n : \{0,1\}^n \to \{0,1\}$ is the restriction of F to inputs of length n.

Informally, POLYSIZE is the class of functions that can be computed by "polynomial size" circuits, meaning, the size of the smallest circuit to compute F on inputs of length n grows (at most) polynomially in n.

Observation 1. There exists a polynomial-time algorithm which, when given a circuit C over n input bits, and an input $x \in \{0,1\}^n$, outputs $C(x)$, the result of running C on input x.

The algorithm just evalutes one gate at a time, from the bottom to the top. If the circuit has size s this takes $O(s \log s)$ time on a RAM, or maybe $O(s^2)$ time on a multitape Turing machine (because we have to scan the circuit back and forth as we are going from the input gates to the output gates).

Definition. For a given computational class C, and a function $\alpha : \mathbb{N} \to \mathbb{N}$, we define a new complexity class C/α, called "**the class C with α bits of advice**" as folows. A function $F : \{0,1\}^* \to \{0,1\}$ is in C/α if there exists some function $a : \mathbb{N} \to \{0,1\}^*$, called an *advice function*, such that:

- forall $n \in \mathbb{N}$, $|a(n)| \leq \alpha(n)$, meaning, the length of the advice for length n is at most $\alpha(n)$, and
- There exists a function $F' \in \mathsf{C}$ such that $\forall x \in \{0,1\}^n \qquad F(x) = F'(x, a(n))$.

Ok, read the definition carefully. It says that for every input length n there exists some "advice $a(n)$ for helping us to compute F on inputs of length n", and the length of this advice is at most $\alpha(n)$.

Definition. We define $\mathsf{P}/poly$ to be the union of P/n^c for all c. I.e. it is the class of problems that can be solved in polynomial time with polynomially-large advice.

Proposition. $POLYSIZE = \mathsf{P}/poly$

Proof. ($\subseteq$) Let F be a problem that can be solved by a family of polynomial-size circuits $C_1, \ldots C_n, \ldots$, i.e., for every $x \in \{0,1\}^n$ we have $C_n(x) = F(x)$, and the size of C_n grows as $O(n^c)$ for some c.

Noticing the observation above: it says that we can evaluate a given circuit. Hence there exists a polynomial-time function F' which on input x and some encoding C_n outputs $C_n(x)$. The "advice" $a(n)$ is exactly a binary encoding of the circuit C_n, and this allows us to compute $F(x)$.

(i.e. $F'(x, D)$ interprets D as encoding a circuit and outputs $D(x)$. This is computable in polynomial time. If given advice $a(n) = C_n$, this computes F.)

($\supseteq$) This is proven by showing that a polynomial-time computation can be computed by a polynomial-size circuit. (Homework) I.e. for any turing machine M, input length n, and time-bound t, there exists a circuit $C_{M,n,t}$ of size $O(t^2 + n)$ that accepts as input a string $x \in \{0,1\}^n$, and outputs the configuration of M when run on input x after t steps.

Now notice the following: if $M(x, a(|x|))$ computes $F(x)$ for all x, then I can *hardwire* the advice $a(n)$ into the circuit. Meaning, the circuit $C_n = C_{m,n+\alpha(n),poly(n)}$ for computing $F(x)$ can be different for each length n. And as part of the circuit I can have constant 0/1 gates.

1. If $F \in P/poly$ then there exists an advice function $a(n)$ and a polytime machine M such that $F(x) = M(x, a(n))$ yes?
2. Now take a circuit $C_n(x, a)$ that simulates M on inputs (x, a) of length $n + n^c$
3. such a circuit exists, yes?
4. From such a circuit I can construct a circuit $C'_n(x) = C(x, a(n))$, simply by hard-wiring the correct advice $a(n)$ onto some of the inputs of C. Do you see?
5. This family C'_n of circuits computes F.

And that is the relationship between polysize circuits and polytime algorithms. For homework you will be asked to prove a similar relationship between circuits of "polynomial width" and PSPACE.

Now some actual lower-bounds!

We have seen only two kinds of lower-bounds so far:

- Hierarchy theorems, that state that having more resources allows us to compute more things.
- Conditional lower-bounds: by showing that $SAT \leq_p A$ for some problem A, it follows that $A \notin P$ unless all of NP is in P.

How about "just showing" the damned lower-bounds, unconditionally? In the second part of the course we will do that.

Recall: for $f : \{0,1\}^n \to \{0,1\}$, $L(f)$ is the smallest number of leaves in a Boolean formula that computes f.

We have seen that most functions have $L(f) \geq \frac{2^n}{\log n}$. We proved this by counting: we count the number of formulas, and the number of functions, and because there are many functions but few small formulas it then followed that most functions do not have small formulas.

But we want to show lower-bounds on $L(f)$ for some *specific* f. It turns out, it is an open problem to prove $L(f) = \omega(n^3)$ for any function f. But we can prove almost cubic lower-bounds. Let's do that now.

Nechiporuk's lower-bound

Consider the following function $f = \textsf{Indexing} \circ \textsf{OR}$.

- Let $n = 2^r$ where r is also a power of 2.
- For a given $z = z_0 \dots z_{r-1} \in \{0,1\}^r$, we interpret it as a number between 0 and $n-1$ written in binary:

$$I(z) = \sum_{i=0}^{r} z_i 2^i \in \{0, \dots, n-1\}$$

- Now consider a function $f : \{0,1\}^n \times \{0,1\}^n \to \{0,1\}$ given as follows.
- The first n-bit input is of the form $x_{1,1}, \dots, x_{r,m}$, where $m = \frac{n}{r}$. I.e. we think of it as an $r \times m$ matrix, where $m = \frac{n}{r}$. So the length of the first input is n bits.
- The second n-bit input is $y_0 \dots y_{n-1}$.
- Then the output $f(x,y) = y_{I(z)}$, where $z_i = x_{i,1} \vee \dots \vee x_{i,m}$. I.e. z_i is the OR of the i-th row of the matrix x.
- In other words, f is a kind of composition of indexing with OR, where the composition appears only in the index, not in the indexed bits y.
- As we have seen, $L(INDEXING_n) = O(n \log n)$, and hence $L(f) = O(n^2 \log n)$.
- We can actually prove that this is nearly tight!:

Theorem (Nechiporuk, 1966). $L(f) = \Omega(\frac{n^2}{\log n \log\log n})$

Proof. First observe that, if we take any Boolean function $h : \{0,1\}^r \to \{0,1\}$, we can set y to be the truth-table of h, meaning, $y = h(0) \dots h(n-1)$, and for this particular fixed choice of y, we have

$$F(x,y) = h(OR(x_{1,1}, \dots, x_{1,m}), \dots, OR(x_{r,1}, \dots, x_{r,m}))$$

This means that, if I have a formula of size S for F, then I can have a formula of size S for the composition of any h with OR.

Now, as we saw last time in class, there exist hard functions h such that $L(h) \geq \frac{2^r}{\log r} = \frac{n}{\log\log n}$.

So let us fix y to be equal to the truth table of such a hard function h.

Ok, let's now do as follows. Let us be given ANY formula φ for $F(x,y)$ with $y = h(0) \dots h(n-1)$.

For each $i \in \{0, \dots, r-1\}$, let x_{i,k_i} be the variable among $x_{i,1}, \dots, x_{i,n-1}$ which appears on the FEWEST leaves of the formula φ.

Obs. we fix every leaf of φ to zero, other than the seleted leaves x_{i,k_i}, and by doing so we get a formula φ' for h. It follows that φ' has at least $L(h)$ leaves because h is hard.

But since x_{i,k_i} is the variable in the i-th block which appears on the fewer number of leaves, the total number of times that variables in the i-th block appear in φ is at least m times that much (there are m variables in each block). Hence

$$L(\varphi) \geq mL(\varphi') \geq \frac{n \cdot m}{\log r} = \Omega(\frac{n^2}{\log n \log\log n})$$

ok?

(As a result of setting some of the leaves to zero, we get a new formula. This formula does not compute the original function. But there is one leaf left in each block, right? and in each block we compute the OR of all variables in that block, right? So when we set all the variables of each block to zero, except for one, the OR now equals the value of that variable which we didn't set to zero. Hence the resulting formula computes $h(x_{1,k_1}, \ldots, x_{r,k_r}))$

Subbotovskaya's lower-bound

Theorem (Subbotovskaya, 1961). $L(\oplus_n) \geq n^{3/2}$.

I.e. formulas for parity must be of size $\Omega(n^{1.5})$.

We had seen that $L(\oplus_n) = O(n^2)$ so the former lower-bound is not tight. For homework you will have to study Hastad's switching lemma, which will improve the 1.5 to 2.

(without the improvement, you could ask "what is the true formula complexity of $\oplus_n$?" and you would know that its a number between $n^{1.5}$ and n^2, but you would not know which, exactly. After the improvement, you know it has quadratic complexity.)

Subbotovskaya's lemma. For every Boolean function f on n variables, it is possible to fix one of its variables in such a way that the resulting function f' on $n-1$ variables satisfies:

$$L(f') \leq (1 - \frac{1}{n})^{3/2} \cdot L(f).$$

In fact, if we choose a variable $i \in [n]$ at random and fix it to a random value 0 or 1, then the above upper-bound holds for $\mathbb{E}[L(f')]$.

To show this, consider some leaf of φ containing a literal $z \in \{x_i, \bar{x}_i\}$. The parent of this leaf is either an AND node or an OR node, so we have a sub-formula:

$$z \wedge G \qquad \text{or} \qquad z \vee G$$

We now claim that because φ is a minimal-size formula, G never includes x_i or $\bar{x}_i$.

Suppose the gate is an AND gate. If $z = x_i$ and x_i appears somewhere in G, we can replace it with 1, and simplify G, because if x_i is zero then the entire formula $z \wedge G$ must be zero, irrespective of the value of G. But this would give me a smaller formula for the same function, contradicting the minimality of φ. The other cases are similar.

Hence, G must contain some leaf with a literal x_j or $\bar{x}_j$ with $j \neq i$. For one of the two settings of the value of x_i, the entire subtree G is redundant, and disappears, causing this extra literal x_j to disappear also. I.e., the leaf containing x_i always disappears, and the other leaf x_j disappears with probability $1/2$. In total, on the expected number of leaves that disappear, over the choice of the value for x_i, is $\frac{3}{2}$. Furthermore, if x_i appears in t_i different leaves, this happens for every

single one of them. So when setting the variable x_i to a random value 0 or 1, the number of leaves of φ decreases by $\frac{3}{2}t_i$ on expectation. In symbols

$$\mathop{\mathbb{E}}_{b}[L(f|_{x_i=b})] \leq L(f) - \frac{3}{2}t_i$$

Hence

$$\mathop{\mathbb{E}}_{i,b}[L(f|_{x_i=b})] \leq L(f) - \frac{3}{2}\mathop{\mathbb{E}}_{i}[t_i] = L(f) - \frac{3}{2}\frac{L(f)}{n}$$

(because there are exactly n variables, so the average t_i is $|\varphi|/n$)

The last value is $\leq (1-\frac{1}{n})^{3/2}L(f)$. tpc

Corollary. Let us take any Boolean function f on n variables. Choose $n-k$ variables uniformly at random, and set them independently and randomly to 0 or 1. This gives us some function f' with only k free variables.

Then, with probability $\geq 3/4$,

$$L(f') \leq 4\left(\frac{k}{n}\right)^{3/2} L(f)$$

Proof. From subbotovskaya's lemma it follows that

$$\mathbb{E}[\mathsf{L}(f')] \leq \cdots \leq \left(1-\frac{1}{n}\right)^{\frac{3}{2}}\left(1-\frac{1}{n-1}\right)^{3/2}\cdots\left(1-\frac{1}{k+1}\right)^{3/2}\mathsf{L}(f) = \left(\frac{k}{n}\right)^{\frac{3}{2}}\mathsf{L}(f).$$

And now the result follows from Markov's inequality.

Proof of Subbotovskaya's theorem. Let $f = \oplus_n$. If we fix $n-1$ uniformly-chosen variables, the resulting f' is still not constant, because the parity function cannot be made constant by fixing any number less than n variables. So $1 \leq L(f')$ always. On the other hand, applying the corollary with $k=1$, we conclude that

$$L(f') \leq 4n^{-3/2}L(\oplus_n)$$

happens with probability $\geq 3/4$. It follows that $L(\oplus) \geq \frac{1}{4}n^{3/2}$.

Ok! Many years later, Hastad has shown that the corollary to subbotovskaya's lemma holds with the exponent 2 replacing 3/2, possibly with a constant worse than 4. It follows from his results that $L(\oplus_n) = \Omega(n^2)$.

Andreev's lower-bound

Remember we looked at Indexing composed with OR. Now let us look at indexing composed with XOR.

- Consider a function $f : \{0,1\}^n \times \{0,1\}^n \to \{0,1\}$ given as follows.
- The first n-bit input is of the form $x_{1,1}, \ldots, x_{r,m}$, where $m = \frac{n}{r}$. I.e. we think of it as an $r \times m$ matrix, where $m = \frac{n}{r}$. So the length of the first input is n bits.
- The second n-bit input is $y_0 \ldots y_{n-1}$.

- Then the output $f(x,y) = y_{I(z)}$, where $z_i = x_{i,1} \oplus \cdots \oplus x_{i,m}$. I.e. z_i is the XOR of the i-th row of the matrix x.

Theorem (Andreev, 1987). $L(f) = \Omega((\frac{n}{\log\log n})^{5/2})$

And with Hastad's improvement 5/2 becomes 3.

Proof. As before, let us fix y to be the truth table of some hard function h with $L(h) \geq \frac{n}{\log n}$.

Let $H(x) = f(x,y)$ for this fixed y.

The idea is: we fix the variables in the blocks at random, so that two things happen simultaneously:

- The formula complexity of f has shrunk by a lot (this uses Subbokovskaya's lemma)
- Every block still has at least one surviving variable, which allows us to use Nechiporuk's argument.

To guarantee both these things simultaneously, we need to carefully choose the probability of fixing a variable.

After some thought, we set $k = 1 + \lceil r\ln(4r) \rceil$, and then choose $n-k$ variables among $x_{1,1}, \ldots, x_{r,m}$, chosen uniformly at random, and fix those variables randomly. Let H' be the resulting function on the k remaining variables (I.e. H' is f where we fixed $y = h$ and we fixed $n-k$ variables among the x).

By subbo...'s lemma, the then know that with probability at least 3/4,

$$L(H') \leq 4\left(\frac{k}{n}\right)^{3/2} L(H).$$

On the other hand, what is the probability that we fixed every variable in one of the blocks?

For a fixed $x_{i,j}$, the probability that $x_{i,j}$ stays free, i.e. is not fixed, is exactly

$$\Pr[x_{i,j} \text{ stays free}] = \frac{\binom{n-1}{k}}{\binom{n}{k}} = \frac{k}{n}$$

Conditioned on the event that some other x variables have already been fixed, the probability that $x_{i,j}$ stays free can only increase. It follows that, for a given block of variables $i \in \{0, \ldots, r-1\}$, the probability that EVERY variable in that block has been fixed is at most:

$$\Pr[x_{i,1} \ldots x_{i,m} \text{ have all been fixed}] \leq \left(1 - \frac{k}{n}\right)^m \leq \exp(-\frac{km}{n}) < \frac{1}{4r}$$

(the number k was chosen precisely to get $\frac{1}{4r}$ on the rigth hand side of this calculation.)

It now follows, by a union bound, that the probability that there *exists* a block i where all the variables have been fixed, is strictly less than $1/4$ (because there are only r blocks). HENCE, there must exist some choice of variables and fixing for them that satisfies both conditions at once:

- Every block has at least one free variable,
- $L(H') \leq 4\left(\frac{k}{n}\right)^{3/2} L(H)$.

But if every block has at least one free variable, then $L(H') \geq L(h) \geq \frac{n}{\log n}$

Putting it together, we conclude:

$$L(f) \geq L(H) \geq \frac{n^{5/2}}{k^{3/2} \log n}$$

WHAT???? YOU'RE TELLING ME THAT THE IDIOTS OF COMPUTATIONAL COMPLEXITY HAVEN'T PROVEN A BETTER LOWER BOUND THAN CUBIC AGAINST BOOLEAN FORMULAS?

yes, that's right. this can probably be explained by the fall of the soviet union.

WHY?

Well, if you ask me why we are stuck precisely at exponent 3, instead of 4 or 5 or so, I'm not really sure.

But there exists an argument that can be made to the effect that: to prove a super-polynomial lower-bound on formula size of an explicit function f, one would need to use proof techniques VERY DIFFERENT from the ones that appeared in the above three proofs.

Hopefully we will have time this year to learn the "natural proofs barrier", which is a meta-result in complexity theory. It states something along the lines of: if cryptography is possible to do in a given computational model, then it is very hard to prove lower-bounds against that model.

And Boolean formulas of polynomial size *are* powerful enough to implement cryptographic primitives.

I'm not sure how large must these cryptography-implementing formulas be, but I would guess... n^3...?

If that is the exact size above which formulas can implement cryptographic primitives, then that is a good explanation why we are stuck at cubic bounds.

Ok that's it.

(Luis asks if there's anywhere where people have tried to show cubic upper bounds on formula size of cryptographic primitives. I don't think so, but I do think that would be an excellent MSc project.

Usually the issue is that cryptographic primitives can get quite complicated, and build on other ones, and so on... But it's worth a try.

)